FULL NAME:	[Enter full name]
DATE:	[Enter submission date]
REGISTRATION NUMBER:	[Enter registration number]
COURSE:	COS 202 (Programming II)

1.0 SMART CLINIC APPLICATION PROJECT REPORT

1.1 PROJECT DESCRIPTION AND FEATURES

The Smart Clinic Appointment and Patient Management System (SCAPMS 3.0) is a C++17 desktop application for the COS 202 lab assessment. Its Qt Widgets interface provides patient registration, appointment management, a FIFO queue, searching, sorting, reporting and persistent storage. Eight panels separate overview, patient records, appointments, queue, algorithms, reports, staff activity and resource calendars.

Scheduled and walk-in visits retain consultation notes. Check-in joins the FIFO queue, Call next starts consultation, and completion saves notes. Booking checks patient, clinician and room conflicts. Cancellation and no-show require reasons, while corrections retain prior notes. Reports show most frequently attended patients, pending appointments, patient category distribution and daily appointment totals. History, archive/restore and undo support record administration.

1.2 CLASS HIERARCHY AND RESPONSIBILITIES

PatientRecord is abstract, with pure virtual category(), consultationSummary() and clone() operations. GeneralPatient, EmergencyPatient and FollowUpPatient provide category-specific behaviour. Appointable defines scheduling and cancellation policies. Processing a PatientRecord reference dispatches to the concrete subtype at runtime, while clone() preserves that subtype in undo snapshots.

ClinicDatabase owns patient and appointment collections. ClinicManager applies validation, permissions and transactions. Separate panels present data; FileHandler and RecoveryService handle persistence. WorkspaceGuard detects competing file access. AccessControl and PortableBackup provide authentication and encryption. ReportService and Benchmark calculate reports and algorithm comparisons. Header/source files and namespaces separate these responsibilities.

1.3 DATA STRUCTURES

A std::vector of unique_ptr<PatientRecord> owns patients without object slicing. Appointments refer to immutable patient IDs and optional clinician/room names. A std::queue of QueueEntry objects preserves arrival order and links waiting patients to visits. A fixed std::array of ten CacheEntry objects tracks frequent access, replacing the least-used entry with an oldest-access tie-break.

A std::stack of UndoSnapshot objects stores up to fifty deep snapshots. Candidate transactions are validated before committing; rejected changes preserve live state and undo history. Undo restores the latest record state while retaining existing audit events and adding an Undo entry. Account changes and switching users clear the undo stack.

1.4 SEARCHING SORTING AND RECURSION

The required searches and sorts are implemented manually. The GUI offers sort selection by patient name, ID or age. Binary search prepares an ID-sorted view with merge sort, costing O(n log n), before the O(log n) lookup.

Algorithm	Time complexity	Purpose and selection reason
Linear search	O(n)	Scans unsorted IDs and name/category filters.
Binary search	O(log n) lookup	Efficient exact ID lookup on the sorted view.
Insertion sort	Best O(n), worst $O(n^2)$	Small or nearly sorted lists. In-place and stable.
Merge sort	O(n log n)	Predictable performance for larger lists. Stable, with O(n) temporary storage.

Merge sort supplies the recursive component. A range with fewer than two items is the base case. Recursive calls sort both halves before merging, preserving equal-key order.

The recorded 2,000-record shuffled-input comparison gives insertion sort a median of 46.287 ms and merge sort 1.735 ms. Each excludes one warm-up, uses five measured runs and excludes input copying. Comparisons were 985,051 and 19,378 respectively. The same input is used for both algorithms, also tested in sorted and reversed order. Timings depend on the test computer. Evidence is in output/screenshots/10-algorithm-comparison.png.

1.5 GUI DESIGN AND EVENTS

VBox and HBox arrange the page and action rows. Grid aligns booking controls, while Form pairs labels with fields. These layouts resize controls consistently. A QAbstractTableModel adapter presents patients. Signals and slots handle actions, selections and timers. Dynamic subtype fields, date/resource filters, duplicate warnings, validation messages, role-aware tooltips, printing and recovery dialogs support daily workflows. Text scaling, visible empty states and keyboard shortcuts improve usability. Search, sort, selection and scrolling survive updates.

1.6 VALIDATION SECURITY AND PERSISTENCE

ValidationError rejects invalid contacts, duplicate IDs, invalid ages, overlapping bookings and invalid visit transitions. FileError reports unreadable, malformed or unwritable workspace files. Both derive from ClinicError. GUI try/catch handlers show a meaningful warning and refresh the view. Candidate changes are validated before commit, so a rejected operation preserves the prior database.

Administrator, clinician, receptionist and viewer roles control editing, clinical details and exports. Passwords are salted and hashed. Windows DPAPI protects saved workspaces and recovery copies. Portable backups use passphrase-based authenticated encryption. Security implementation details are documented in docs/security-and-recovery.md.

QSaveFile commits writes atomically. WorkspaceGuard reserves a canonical path with QLockFile and checks a SHA-256 fingerprint before saving. Autosave runs every thirty seconds, with unique files for unsaved sessions. Manual saves retain five protected previous versions. Recovery previews show timestamps and record counts; imported or recovered data opens unsaved for Save As.

1.7 CHALLENGES AND RESOLUTIONS

Undo initially required copying records held by unique_ptr. Virtual clone() now creates deep snapshots that retain each patient's subtype. Undo restores the last administrative state while preserving audit events. Tests verify subtype preservation and rollback after rejected changes.

An empty-state callback accessed a table during destruction and caused a GUI teardown failure. Parenting the helper to the viewport and guarding object references with QPointer resolved the failure. The full core and GUI suite subsequently passed. Walk-ins also received persistent visit records so completed notes remain visible in history.

1.8 VERIFICATION

The final Qt Test run reports 31 results, 0 failures and 0 errors. Validation, workflow, permissions, persistence, recovery, algorithms and GUI behaviour are covered. Backup transfer to a second computer remains unverified.

1.9 PROJECT REPOSITORY

https://github.com/atanunu/COS-202-SCAPMS

The public repository contains src/ for C++ headers and implementation, data/ for fictional input files, tests/ for automated checks, scripts/ for build tools, output/screenshots/ for GUI evidence and docs/diagrams/ for UML source, PNG and PDF. The assessor can open these materials without signing in.

Assessed implementation version 63fb938 (fixed source snapshot)